1. Conceptual & Logical E/R Diagrams

Exercise 1.1: Conceptual E/R Diagram

Scenario: A small university needs to manage information about students, courses, and instructors.
Task:

- Identify the main entities (e.g., Student, Course, Instructor).
- List the key attributes for each entity (e.g., StudentID, Name, Major for Student).
- Define relationships between these entities (e.g., a Student enrolls in Courses; an Instructor teaches Courses).
- Specify cardinalities (one-to-many, many-to-many, etc.) and participation constraints.

Questions to Consider:

- What are the primary keys for each entity?
- How do you represent many-to-many relationships conceptually?

Exercise 1.2: Logical E/R Diagram

Task:

- Convert your conceptual E/R diagram from Exercise 1.1 into a logical E/R diagram.
- Add details such as primary keys (PKs), foreign keys (FKs), and any necessary relationship tables (especially for many-to-many relationships).
- Draw the diagram using any diagramming tool or on paper.
- Provide a brief explanation of your choices (e.g., why you selected certain attributes as keys).

---

2. Physical E/R Diagram

Exercise 2.1: Designing a Physical Schema

Scenario: Use the same university example.
Task:

- Translate your logical E/R diagram into a physical database design.
- Create table definitions that include columns, data types, primary keys, foreign keys, and any indexing decisions.
- Write the SQL CREATE TABLE statements for each table.

Questions to Consider:

- How do you optimize the physical design for query performance?

- Which fields are most likely to be indexed and why?

---

3. Normal Forms (1st, 2nd, and 3rd)

Exercise 3.1: Normalization Theory

Task:

- Define 1st, 2nd, and 3rd normal forms (1NF, 2NF, 3NF).
- Provide an example of a table structure that violates each form.
- Explain why the table violates the specific normal form and what anomalies might result.

Exercise 3.2: Normalization in Practice

Scenario: Consider the following unnormalized table representing orders:

OrderID	CustomerName	CustomerAddress	Product1	Product2	Product3	OrderDate
1001	Alice	123 Main St	Widget A	Widget B	(null)	2025-03-01

Task:

- Identify the issues in this table (e.g., repeating groups, potential update anomalies).
- Transform the table into 1NF by eliminating repeating groups.
- Further decompose the resulting tables to achieve 2NF and 3NF.
- Explain each step and the rationale behind your design choices.

---

4. Data Redundancy and Update Anomalies

Exercise 4.1: Identifying Anomalies

Task:

- Describe the three types of update anomalies: insertion, deletion, and modification.
- Provide a brief example for each anomaly type using a simple table (for instance, a table that includes employee and department data).

Exercise 4.2: Resolving Anomalies

Scenario: A company maintains a single table for employee data:

EmpID	EmpName	Department	DeptManager
001	John	Sales	Karen
002	Mary	Sales	Karen

EmpID	EmpName	Department	DeptManager
003	Steve	IT	Bob

Task:

- Identify the potential update anomalies (for example, if the manager for Sales changes, multiple rows must be updated).
- Redesign the schema to eliminate redundancy. Create separate tables (e.g., one for Employees and one for Departments).
- Write the SQL statements to create the normalized tables and explain how your design avoids update anomalies.

---

5. Mixed Exercise: End-to-End Database Design

Exercise 5.1: Complete Database Design

Scenario: A company database includes Employees, Departments, and Projects. Employees can work on multiple projects, and projects can have multiple employees. Departments manage projects and have multiple employees.

Task:

- Create a conceptual E/R diagram that identifies entities, attributes, and relationships.
- Develop a logical E/R diagram by adding keys and detailing relationships.
- Convert the logical diagram into a physical schema with SQL CREATE TABLE statements.
- Identify any normalization issues in your design and describe how you ensured your tables meet 3NF.
- Discuss any data redundancy issues that could lead to update anomalies and explain how your design prevents them.